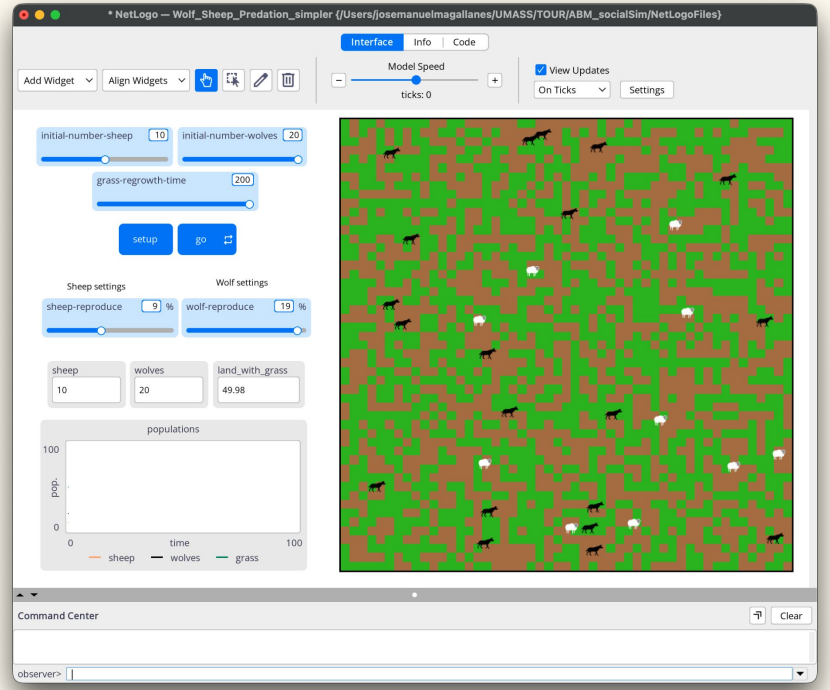
Scenarios

Use code:
Wolf_Sheep_Predation_simpler.nlogox

The case: Wolf and Sheep



HOW IT WORKS

- Wolves and sheep wander randomly around the landscape. The take one “step” at a time.
- Each “step” costs the sheep energy.
- Sheep’s energy can be replenish when they eat grass.
- The sheep eat grass if available.
- Sheep die when they run out of energy. They couldn’t find enough grass.
- Each “step” costs the wolves energy.
- Wolves’ energy can be replenish when they eat sheep.
- The wolves look for sheep to prey on.
- Wolves die when they run out of energy. They couldn’t find enough sheep.
- To allow the population to continue, each wolf or sheep has a fixed probability of reproducing at each time step.
- Once grass is eaten it will regrow after a fixed amount of time.

The case: Wolf and Sheep

```
to go

  if not any? turtles [stop] ; stop the model if there are no wolves nor sheep

  ask sheep  [
    wander
    reduce-energy; sheep lose energy as they move
    eat-grass  ; sheep eat grass (if available at their location)
    timeToDie ; sheep die from starvation (no energy left)
    reproduce-sheep  ; sheep reproduce at a random rate governed by a slider
  ]

  ask wolves [
    wander
    reduce-energy ; wolves lose energy as they move
    eat-sheep ; wolves eat (if available at their location)
    timeToDie ; wolves die from starvation (no energy left)
    reproduce-wolves ; wolves reproduce at a random rate governed by a slider
  ]

  ask patches [
    re-grow-grass
  ]

  set current_grass count patches with [pcolor = green] ; for reporter
  tick

end
```

HOW IT WORKS

- Wolves and sheep wander randomly around the landscape. The take one “step” at a time.
- Each “step” costs the sheep energy.
- Sheep’s energy can be replenish when they eat grass.
- The sheep eat grass if available.
- Sheep die when they run out of energy. They couldn’t find enough grass.
- Each “step” costs the wolves energy.
- Wolves’ energy can be replenish when they eat sheep.
- The wolves look for sheep to prey on.
- Wolves die when they run out of energy. They couldn’t find enough sheep.
- To allow the population to continue, each wolf or sheep has a fixed probability of reproducing at each time step.
- Once grass is eaten it will regrow after a fixed amount of time.

The case: Wolf and Sheep

```
;;;;turtle procedures (i.e. both wolf and sheep procedure)
to wander
  rt random 360
  fd 1
end

to reduce-energy
  set energy energy - 1
end

to timeToDie
   if energy < 0 [ die ]
end
```

Common procedures

HOW IT WORKS

- Wolves and sheep wander randomly around the landscape. The take one “step” at a time.
- Each “step” costs the sheep energy.
- Sheep’s energy can be replenish when they eat grass.
- The sheep eat grass if available.
- Sheep die when they run out of energy. They couldn’t find enough grass.
- Each “step” costs the wolves energy.
- Wolves’ energy can be replenish when they eat sheep.
- The wolves look for sheep to prey on.
- Wolves die when they run out of energy. They couldn’t find enough sheep.
- To allow the population to continue, each wolf or sheep has a fixed probability of reproducing at each time step.
- Once grass is eaten it will regrow after a fixed amount of time.



The case: Wolf and Sheep

SHEEP procedures



HOW IT WORKS

- Wolves and sheep wander randomly around the landscape. The take one “step” at a time.
- Each “step” costs the sheep energy.
- Sheep’s energy can be replenish when they eat grass.
- The sheep eat grass if available.
- Sheep die when they run out of energy. They couldn’t find enough grass.
- Each “step” costs the wolves energy.
- Wolves’ energy can be replenish when they eat sheep.
- The wolves look for sheep to prey on.
- Wolves die when they run out of energy. They couldn’t find enough sheep.
- To allow the population to continue, each wolf or sheep has a fixed probability of reproducing at each time step.
- Once grass is eaten it will regrow after a fixed amount of time.

```
;;;;sheep-only procedures
to eat-grass
  if pcolor = green [
    set pcolor brown
    set energy energy + sheep-gain-from-food  ; sheep eat grass and increase their energy
  ]
end

to reproduce-sheep
  if random-float 100 < sheep-reproduce [  ; throw "dice" to see if you will reproduce
    set energy (energy / 2)                ; divide energy between parent and offspring
    hatch 1 [                     ; hatch offspring, which inherits  half the energy of mother
      rt random-float 360 fd 1 ]   ; hatched offspring moves forward 1 step
  ]
end
```

The case: Wolf and Sheep

WOLVES procedures



HOW IT WORKS

- Wolves and sheep wander randomly around the landscape. The take one “step” at a time.
- Each “step” costs the sheep energy.
- Sheep’s energy can be replenish when they eat grass.
- The sheep eat grass if available.
- Sheep die when they run out of energy. They couldn’t find enough grass.
- Each “step” costs the wolves energy.
- Wolves’ energy can be replenish when they eat sheep.
- The wolves look for sheep to prey on.
- Wolves die when they run out of energy. They couldn’t find enough sheep.
- To allow the population to continue, each wolf or sheep has a fixed probability of reproducing at each time step.
- Once grass is eaten it will regrow after a fixed amount of time.

```
;;;;wolves-only procedures

to reproduce-wolves
  if random-float 100 < wolf-reproduce [  ; throw "dice" to see if you will reproduce
    set energy (energy / 2)               ; share your energy with offspring
    hatch 1 [ rt random-float 360 fd 1 ]  ; hatched offspring moves forward 1 step
  ]
end

to eat-sheep
  let prey one-of sheep-here                    ; grab a random sheep (which is here in my patch)
  if prey != nobody  [                          ; did we get one? if so,
    ask prey [ die ]                            ; kill it, and...
    set energy energy + wolf-gain-from-food     ; get energy from eating
  ]
end
```

The case: Wolf and Sheep

PATCHES procedures



HOW IT WORKS

- Wolves and sheep wander randomly around the landscape. The take one “step” at a time.
- Each “step” costs the sheep energy.
- Sheep’s energy can be replenish when they eat grass.
- The sheep eat grass if available.
- Sheep die when they run out of energy. They couldn’t find enough grass.
- Each “step” costs the wolves energy.
- Wolves’ energy can be replenish when they eat sheep.
- The wolves look for sheep to prey on.
- Wolves die when they run out of energy. They couldn’t find enough sheep.
- To allow the population to continue, each wolf or sheep has a fixed probability of reproducing at each time step.
- Once grass is eaten it will regrow after a fixed amount of time.

```
;;;;patches-only procedures
to re-grow-grass

  if pcolor = brown [ ; if you have not food
    ifelse countdown <= 0 ; if your clock is zero or less
      [ set pcolor green ; become food
        set countdown grass-regrowth-time ]  ; restart clock
      [ set countdown countdown - 1 ] ; if you are not food and clock greater than zero
  ]

end
```

The case: Wolf and Sheep

using
REPORTERS

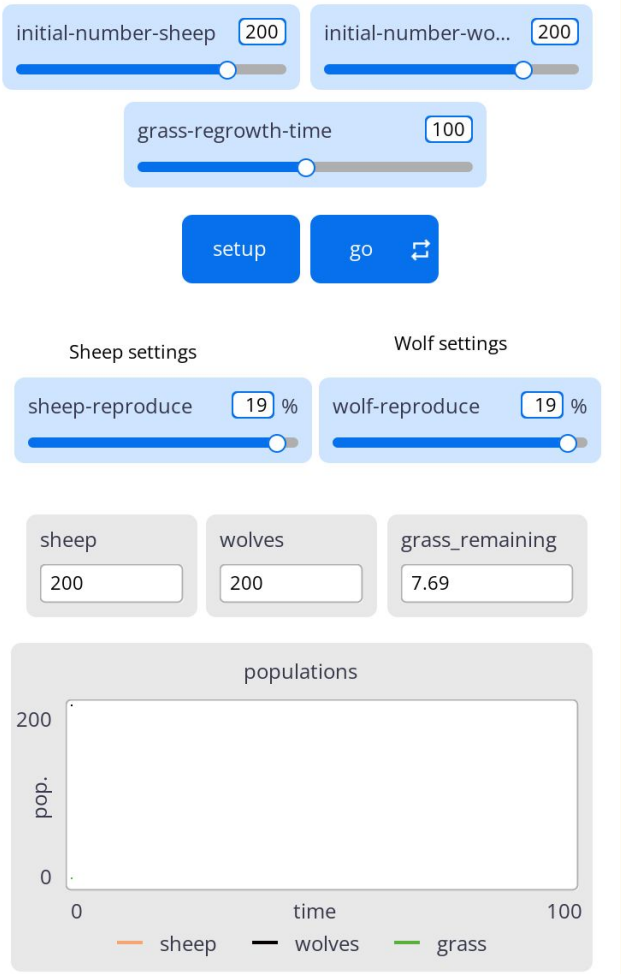


HOW IT WORKS

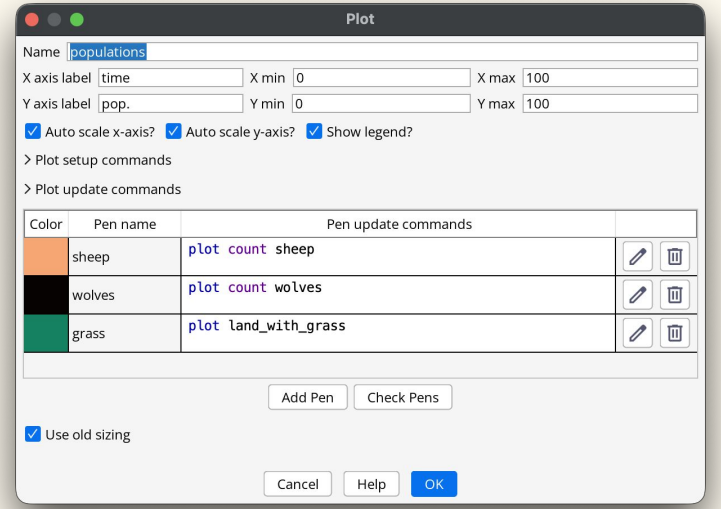
- Wolves and sheep wander randomly around the landscape. The take one “step” at a time.
- Each “step” costs the sheep energy.
- Sheep’s energy can be replenish when they eat grass.
- The sheep eat grass if available.
- Sheep die when they run out of energy. They couldn’t find enough grass.
- Each “step” costs the wolves energy.
- Wolves’ energy can be replenish when they eat sheep.
- The wolves look for sheep to prey on.
- Wolves die when they run out of energy. They couldn’t find enough sheep.
- To allow the population to continue, each wolf or sheep has a fixed probability of reproducing at each time step.
- Once grass is eaten it will regrow after a fixed amount of time.

You will get a VALUE when you call a “reporter”

```
;;;;REPORTERS
to-report land_with_grass
  report 100 * count patches with[pcolor = green] / count patches
end
```



You will get a VALUE when you call a "reporter"





```
;;;;REPORTERS
to-report land_with_grass
  report 100 * count patches with[pcolor = green] / count patches
end
```

Scenario Analysis

Example - Using BehaviorSpace: the 100/100 and 400/400 experiments

INPUT variables:
Vary “sheep-reproduce” from 4, in increments of 2, to 20.
Vary “wolf-reproduce” from 4, in increments of 2, to 20.
Vary “grass-regrowth-time” from 150, in increments of 25, to 250.

Fixed variables (set in Setup commands, before calling setup):
For the “100/100” experiment: set initial-number-sheep to 100, set initial-number-wolves to 100.
For the “400/400” experiment: set initial-number-sheep to 400, set initial-number-wolves to 400.

OUTPUT variables:
You want to know how all those variations relate to “land_with_grass”, “count sheep”, and “count wolves”.

Repetitions: 3.
Run metrics every step: off (only the final value of each run is recorded).

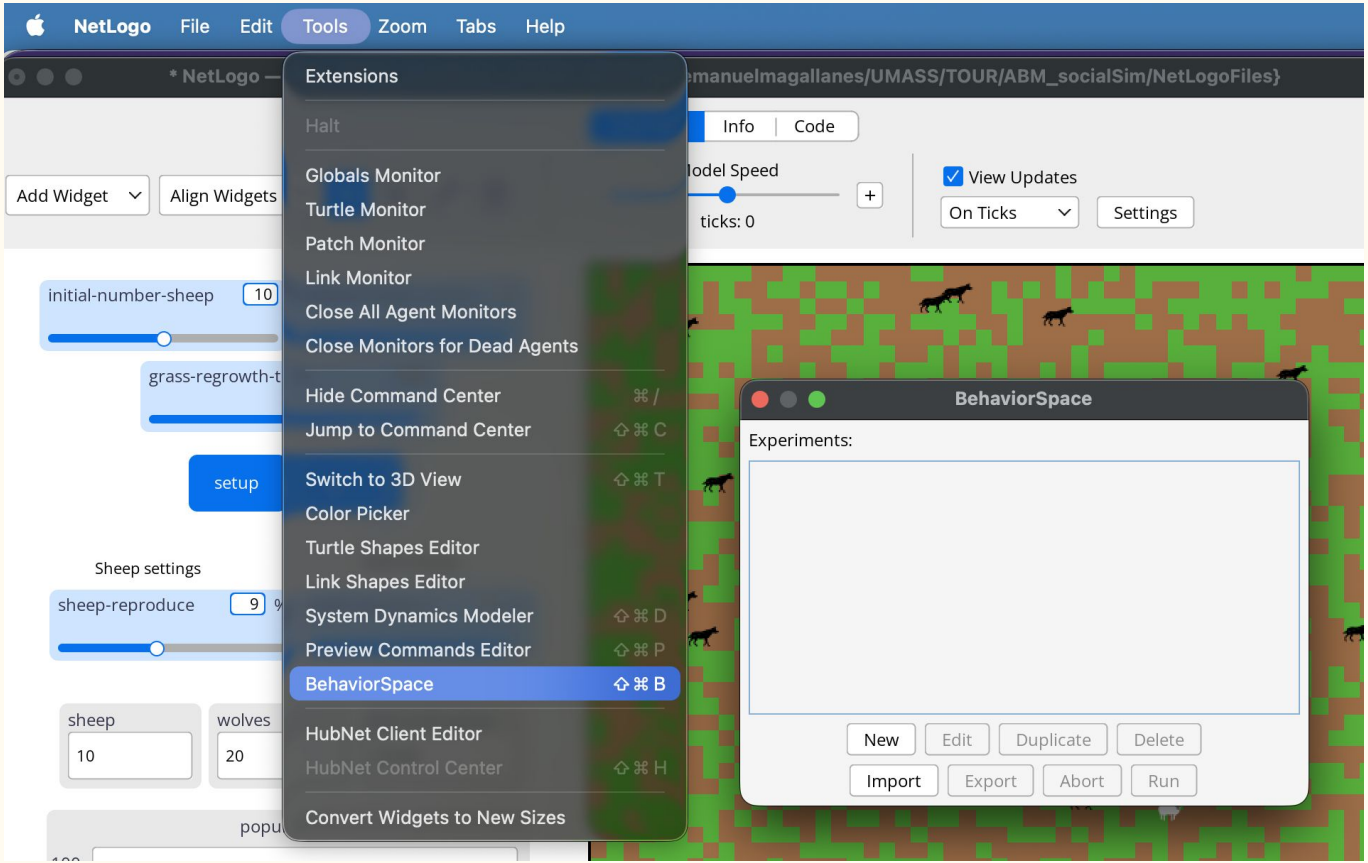
STOP condition:
Use: (land_with_grass = 0) or (not any? turtles) as STOP condition.

Time limit: 1500.

When each experiment finishes, export its table of results — these are the two files (“experiment_100_100-table.csv” and “experiment_400_400-table.csv”) used to compare the two population sizes. I will upload those files to GitHub to run post analysis using Python in Colab.

IMPORTANT FOR THE HOMEWORK

Scenario Analysis

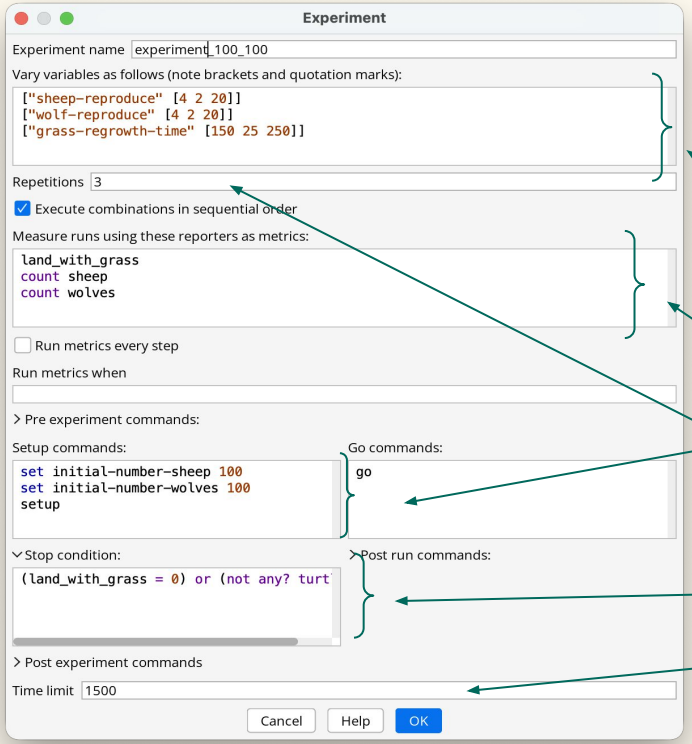


Using:

Behavior Space

Scenario Analysis

Using:
Behavior
Space



Example - Using BehaviorSpace: the 100/100 and 400/400 experiments

INPUT variables:
Vary “sheep-reproduce” from 4, in increments of 2, to 20.
Vary “wolf-reproduce” from 4, in increments of 2, to 20.
Vary “grass-regrowth-time” from 150, in increments of 25, to 250.

Fixed variables (set in Setup commands, before calling setup):
For the “100/100” experiment: set initial-number-sheep to 100, set initial-number-wolves to 100.
For the “400/400” experiment: set initial-number-sheep to 400, set initial-number-wolves to 400.

OUTPUT variables:
You want to know how all those variations relate to “land_with_grass”, “count sheep”, and “count wolves”.

Repetitions: 3.
Run metrics every step: off (only the final value of each run is recorded).

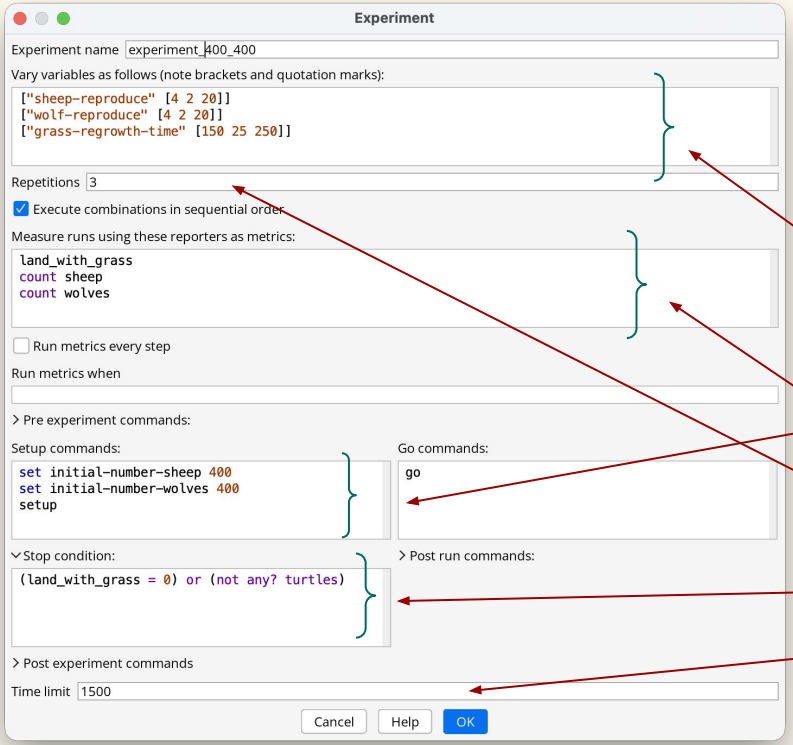
STOP condition:
Use: (land_with_grass = 0) or (not any? turtles) as STOP condition.

Time limit: 1500.

When each experiment finishes, export its table of results — these are the two files (“experiment_100_100-table.csv” and “experiment_400_400-table.csv”) used to compare the two population sizes. I will upload those files to GitHub to run post analysis using Python in Colab.

Scenario Analysis

Using:
Behavior
Space



Example - Using BehaviorSpace: the 100/100 and 400/400 experiments

INPUT variables:
Vary “sheep-reproduce” from 4, in increments of 2, to 20.
Vary “wolf-reproduce” from 4, in increments of 2, to 20.
Vary “grass-regrowth-time” from 150, in increments of 25, to 250.

Fixed variables (set in Setup commands, before calling setup):
For the “100/100” experiment: set initial-number-sheep to 100, set initial-number-wolves to 100.
For the “400/400” experiment: set initial-number-sheep to 400, set initial-number-wolves to 400.

OUTPUT variables:
You want to know how all those variations relate to “land_with_grass”, “count sheep”, and “count wolves”.

Repetitions: 3.
Run metrics every step: off (only the final value of each run is recorded).

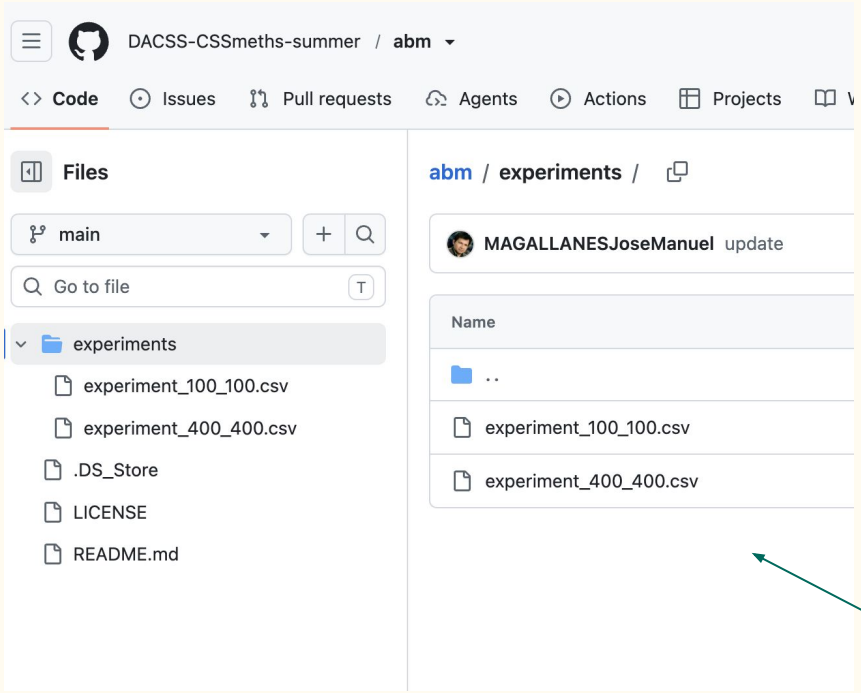
STOP condition:
Use: (land_with_grass = 0) or (not any? turtles) as STOP condition.

Time limit: 1500.

When each experiment finishes, export its table of results — these are the two files (“experiment_100_100-table.csv” and “experiment_400_400-table.csv”) used to compare the two population sizes. I will upload those files to GitHub to run post analysis using Python in Colab.

Scenario Analysis

Using:
Behavior
Space



Example - Using BehaviorSpace: the 100/100 and 400/400 experiments

INPUT variables:
Vary "sheep-reproduce" from 4, in increments of 2, to 20.
Vary "wolf-reproduce" from 4, in increments of 2, to 20.
Vary "grass-regrowth-time" from 150, in increments of 25, to 250.

Fixed variables (set in Setup commands, before calling setup):
For the "100/100" experiment: set initial-number-sheep to 100, set initial-number-wolves to 100.
For the "400/400" experiment: set initial-number-sheep to 400, set initial-number-wolves to 400.

OUTPUT variables:
You want to know how all those variations relate to "land_with_grass", "count sheep", and "count wolves".

Repetitions: 3.
Run metrics every step: off (only the final value of each run is recorded).

STOP condition:
Use: (land_with_grass = 0) or (not any? turtles) as STOP condition.

Time limit: 1500.

When each experiment finishes, export its table of results — these are the two files ("experiment_100_100-table.csv" and "experiment_400_400-table.csv") used to compare the two population sizes. I will upload those files to GitHub to run post analysis using Python in Colab.

Scenario Analysis

Results from the Behavior Space experiments will need to be analyzed OUTSIDE NetLogo

VISIT THE **COLAB** NOTEBOOK LINK PROVIDED

Wolf Sheep Predation — Comparing Two BehaviorSpace Runs

Comparing **100 sheep / 100 wolves** vs **400 sheep / 400 wolves** — same model code (unmodified), same parameter grid (`sheep-reproduce` , `wolf-reproduce` , `grass-regrowth-time`), same time limit (1500 ticks).

Data links:

[1] ✓ 0s

```
bs100link='https://github.com/DACSS-CSSmeths-summer/abm/raw/main/Wolf_Sheep_Predation_experiment_100_100-table.csv'
bs400link='https://github.com/DACSS-CSSmeths-summer/abm/raw/main/Wolf_Sheep_Predation_experiment_400_400-table.csv'
```

You will bring another file, and compare it to ONE of these

[2] ✓ 1s

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

pd.set_option("display.max_columns", None)
```

1.- Load the data

BehaviorSpace table exports start with 6 lines of metadata (model name, timestamp, world dimensions) before the actual header row, so we skip them with `skiprows=6` . We also rename the columns to something easier to work with in code — the raw column names come out as things like `[run number]` and `sheep-reproduce` , which are awkward to reference in pandas.

[3] ✓ 0s

```
def read_bs_table(path):
    df = pd.read_csv(path, skiprows=6)
    df.columns = ["run_number", "sheep_reproduce", "wolf_reproduce",
                  "grass_regrowth_time", "step",
                  "land_with_grass", "count_sheep", "count_wolves"]
    return df

exp_100 = read_bs_table(bs100link)
exp_400 = read_bs_table(bs400link)
```